

THE COMPLETE SQL REFERENCE

Rules Clauses Functions Joins Window Functions + 50 LeetCode SQL Solutions

Generated May 30, 2026 | Standard ANSI SQL / MySQL / PostgreSQL Syntax

TABLE OF CONTENTS

PART 1 - SQL FUNDAMENTALS

1. Data Definition Language (DDL): CREATE, ALTER, DROP, TRUNCATE
2. Data Manipulation Language (DML): INSERT, UPDATE, DELETE, MERGE
3. Data Query Language (DQL) - SELECT: FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT
4. Filtering & Conditions: WHERE, AND/OR/NOT, IN, BETWEEN, LIKE, IS NULL
5. Joins: INNER, LEFT, RIGHT, FULL, CROSS, SELF
6. Subqueries & CTEs: Correlated, EXISTS, WITH
7. Set Operations: UNION, INTERSECT, EXCEPT
8. Aggregate Functions: COUNT, SUM, AVG, MIN, MAX, GROUP_CONCAT
9. String Functions: CONCAT, SUBSTRING, LENGTH, REPLACE, TRIM, UPPER/LOWER...
10. Date & Time Functions: NOW, DATE_FORMAT, DATEDIFF, DATE_ADD
11. Numeric & Math Functions: ROUND, FLOOR, CEIL, MOD, ABS, POWER
12. Conditional Expressions: CASE WHEN, IF, IFNULL, COALESCE, NULLIF
13. Window Functions: ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, SUM OVER
14. Indexes & Performance: CREATE INDEX, EXPLAIN, query optimisation tips
15. Transactions & ACID: BEGIN, COMMIT, ROLLBACK, SAVEPOINT
16. Views & Stored Procedures: CREATE VIEW, Stored Procedures, Functions
17. Data Types Reference: INT, VARCHAR, DATE, DECIMAL, JSON ...

PART 2 - 50 LEETCODE SQL SOLUTIONS

Problems 1-10 (Easy): 175, 176, 177, 181, 182, 183, 184, 185, 196, 197

Problems 11-25 (Easy/Medium): 262, 511, 512, 550, 570, 571, 574, 577, 578, 579, 580, 584, 585, 586, 595

Problems 26-40 (Medium): 596, 597, 601, 602, 607, 608, 610, 612, 613, 614, 615, 618, 619, 620, 626

Problems 41-50 (Medium/Hard): 627, 1045, 1050, 1068, 1069, 1070, 1075, 1076, 1082, 1083

PART 1 - SQL FUNDAMENTALS

1. Data Definition Language (DDL)

DDL statements define and modify the structure of database objects (schemas, tables, columns, constraints, indexes). They perform an implicit COMMIT in most databases.

1.1 CREATE TABLE

Creates a new table with specified columns and constraints.

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    salary DECIMAL(10,2) DEFAULT 0.00,  
    dept_id INT,  
    hire_date DATE,  
    CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES departments(dept_id)  
    ON DELETE SET NULL ON UPDATE CASCADE  
);
```

TIP: Always define a PRIMARY KEY. Use UNSIGNED INT/BIGINT for IDs that will never be negative to double the positive range.

1.2 ALTER TABLE

```
-- Add column  
ALTER TABLE employees ADD COLUMN phone VARCHAR(20) AFTER email;  
  
-- Modify column  
ALTER TABLE employees MODIFY COLUMN salary DECIMAL(12,2) NOT NULL DEFAULT 50000.00;  
  
-- Rename column (MySQL 8+)  
ALTER TABLE employees RENAME COLUMN phone TO mobile;  
  
-- Drop column  
ALTER TABLE employees DROP COLUMN mobile;  
  
-- Add constraint  
ALTER TABLE employees ADD CONSTRAINT chk_salary CHECK (salary >= 0);  
  
-- Drop constraint  
ALTER TABLE employees DROP CONSTRAINT chk_salary;
```

1.3 DROP & TRUNCATE

Remove table and all data permanently:

```
DROP TABLE IF EXISTS temp_logs;
```

Remove all rows, reset AUTO_INCREMENT, keep structure:

```
TRUNCATE TABLE audit_log;
```

Drop database:

```
DROP DATABASE IF EXISTS old_project;
```

WARNING: TRUNCATE cannot be rolled back in MySQL. DROP is irreversible. Always backup first.

1.4 CREATE INDEX

Single-column index:

```
CREATE INDEX idx_last_name ON employees(last_name);
```

Composite index (column order matters!):

```
CREATE INDEX idx_dept_salary ON employees (dept_id, salary DESC);
```

Unique index:

```
CREATE UNIQUE INDEX idx_email ON employees(email);
```

Full-text index (MySQL):

```
CREATE FULLTEXT INDEX idx_bio ON profiles (bio);
```

Partial index (PostgreSQL):

```
CREATE INDEX idx_active ON employees (dept_id) WHERE active = TRUE;
```

2. Data Manipulation Language (DML)

DML statements add, modify, or delete data inside tables.

2.1 INSERT

Single row:

```
INSERT INTO employees (first_name, last_name, email, salary, dept_id, hire_date)
VALUES ('Alice', 'Johnson', 'alice@example.com', 85000.00, 3, '2023-06-01');
```

Multiple rows (efficient batch insert):

```
INSERT INTO employees (first_name, last_name, salary, dept_id)
VALUES
('Bob', 'Smith', 72000, 2),
('Carol', 'White', 91000, 1),
('David', 'Brown', 67000, 4);
```

Insert from SELECT (copy data):

```
INSERT INTO employees_archive
SELECT * FROM employees WHERE hire_date < '2020-01-01';
```

INSERT ... ON DUPLICATE KEY UPDATE (MySQL upsert):

```
INSERT INTO page_views (page_id, view_count)
VALUES (101, 1)
ON DUPLICATE KEY UPDATE view_count = view_count + 1;
```

INSERT ON CONFLICT (PostgreSQL upsert):

```
INSERT INTO page_views (page_id, view_count)
VALUES (101, 1)
ON CONFLICT (page_id)
DO UPDATE SET view_count = page_views.view_count + 1;
```

2.2 UPDATE

Update single column:

```
UPDATE employees SET salary = salary * 1.10 WHERE dept_id = 3;
```

Update multiple columns:

```
UPDATE employees
SET salary = 95000, dept_id = 1
WHERE emp_id = 42;
```

Update with JOIN (MySQL):

```
UPDATE employees e
JOIN departments d ON e.dept_id = d.dept_id
SET e.salary = e.salary * 1.05
WHERE d.dept_name = 'Engineering';
```

Safe update: use ORDER BY and LIMIT to prevent mass accidents:

```
UPDATE employees SET salary = 60000
WHERE dept_id = 5
ORDER BY emp_id
LIMIT 11;
```

2.3 DELETE

Delete specific rows:

```
DELETE FROM employees WHERE emp_id = 99;
```

Delete with subquery:

```
DELETE FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE active = 0);
```

Delete duplicates, keep lowest id:

```
DELETE e1
FROM employees e1
JOIN employees e2
ON e1.email = e2.email AND e1.emp_id > e2.emp_id;
```

2.4 MERGE (UPSERT)

SQL Server / Oracle syntax:

```
MERGE INTO employees AS target
USING new_hires AS source
ON target.email = source.email
WHEN MATCHED THEN
    UPDATE SET target.salary = source.salary
WHEN NOT MATCHED THEN
    INSERT (first_name, last_name, email, salary)
    VALUES (source.first_name, source.last_name, source.email, source.salary);
```

3. Data Query Language - SELECT (Full Anatomy)

Understanding the logical execution order of a SELECT is critical for writing correct queries.

Logical Execution Order

Step	Clause	What Happens
1	FROM/JOIN	Identify source tables, build Cartesian product, apply join predicates
2	WHERE	Filter rows (cannot use aliases or aggregates)
3	GROUP BY	Collapse rows into groups
4	HAVING	Filter groups (can use aggregate results)
5	SELECT	Compute expressions, assign aliases
6	DISTINCT	Remove duplicate rows
7	ORDER BY	Sort result (can use aliases from SELECT)
8	LIMIT/OFFSET	Restrict number of rows returned

Full SELECT Skeleton

```
SELECT
    d.dept_name,
```

```

COUNT(e.emp_id) AS total_employees,
AVG(e.salary) AS avg_salary,
MAX(e.salary) AS top_salary
FROM
    employees e
    INNER JOIN departments d ON e.dept_id = d.dept_id
WHERE
    e.hire_date >= '2020-01-01'
GROUP BY
    d.dept_name
HAVING
    COUNT(e.emp_id) > 2
ORDER BY
    avg_salary DESC
LIMIT 10 OFFSET 0;

```

3.1 SELECT Expressions & Aliases

```

SELECT
    emp_id,
    first_name || ' ' || last_name AS full_name, -- PostgreSQL
    CONCAT(first_name, ' ', last_name) AS full_name, -- MySQL
    salary / 12 AS monthly_pay,
    ROUND(salary * 0.15, 2) AS tax,
    salary > 80000 AS is_senior, -- boolean in MySQL
    CASE
        WHEN salary > 100000 THEN 'High'
        WHEN salary > 60000 THEN 'Mid'
        ELSE 'Entry'
    END AS pay_band
FROM employees;

```

3.2 WHERE Clause Deep Dive

```

-- Comparison operators
WHERE salary > 50000
WHERE hire_date BETWEEN '2022-01-01' AND '2022-12-31'
WHERE dept_id IN (1, 2, 3)
WHERE dept_id NOT IN (SELECT dept_id FROM closed_departments)
WHERE email IS NULL
WHERE email IS NOT NULL

-- Pattern matching
WHERE last_name LIKE 'S%'           -- starts with S
WHERE first_name LIKE '%ohn%'       -- contains 'ohn'
WHERE email LIKE '%.com'            -- ends with .com
WHERE last_name LIKE 'm_th'         -- matches exactly one char

-- Regex (MySQL)
WHERE last_name REGEXP '^[A-C]'

```

```
-- Logical operators (precedence: NOT > AND > OR)
WHERE (dept_id = 1 OR dept_id = 2) AND salary > 70000
WHERE NOT (salary BETWEEN 40000 AND 50000)
```

4. JOINS - Complete Reference

Joins combine rows from two or more tables based on a related column.

4.1 INNER JOIN

Returns rows that have matching values in both tables.

```
SELECT e.first_name, e.last_name, d.dept_name
FROM employees e
INNER JOIN departments d ON e.dept_id = d.dept_id;

-- Shorthand syntax
FROM employees e JOIN departments d ON e.dept_id = d.dept_id
```

4.2 LEFT (OUTER) JOIN

Returns all rows from the left table and matching rows from the right. Non-matching right rows are NULL.

```
SELECT e.first_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id;
-- Employees without a department still appear (dept_name = NULL)
```

4.3 RIGHT (OUTER) JOIN

```
SELECT e.first_name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;
-- All departments appear, even those with no employees
```

4.4 FULL OUTER JOIN

All rows from both tables:

```
SELECT e.first_name, d.dept_name
FROM employees e
FULL OUTER JOIN departments d ON e.dept_id = d.dept_id;

-- MySQL Workaround (no native FULL OUTER JOIN)
SELECT e.first_name, d.dept_name FROM employees e LEFT JOIN departments d ON e.dept_id
= d.dept_id
UNION
```

```
SELECT e.first_name, d.dept_name FROM employees e RIGHT JOIN departments d ON  
e.dept_id = d.dept_id;
```

4.5 CROSS JOIN

Cartesian product: every row x every row

```
SELECT c.color, s.size  
FROM colors c  
CROSS JOIN sizes s;  
-- e.g., 5 colors x 4 sizes = 20 rows
```

4.6 SELF JOIN

Find each employee's manager (same table):

```
SELECT e.first_name AS employee, m.first_name AS manager  
FROM employees e  
LEFT JOIN employees m ON e.manager_id = m.emp_id;
```

4.7 Multi-table JOIN

```
SELECT e.first_name, d.dept_name, l.city, j.job_title  
FROM employees e  
JOIN departments d ON e.dept_id = d.dept_id  
JOIN locations l ON d.loc_id = l.loc_id  
JOIN job_roles j ON e.job_id = j.job_id  
WHERE l.country = 'USA';
```

4.8 JOIN with USING

When column name is identical in both tables:

```
SELECT e.first_name, d.dept_name  
FROM employees e  
JOIN departments d USING (dept_id);
```

5. Subqueries & CTEs

5.1 Scalar Subquery

Returns a single value:

```
SELECT first_name, salary,  
       (SELECT AVG(salary) FROM employees) AS company_avg,  
       salary - (SELECT AVG(salary) FROM employees) AS diff  
FROM employees;
```


5.2 Row Subquery in WHERE

```
-- IN subquery
SELECT * FROM employees
WHERE dept_id IN (SELECT dept_id FROM departments WHERE location = 'NYC');

-- NOT IN
SELECT * FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders);

-- Comparison with scalar subquery
SELECT * FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

5.3 Correlated Subquery

Executes once per outer row - useful but can be slow. Finding employees earning above their department average:

```
SELECT first_name, salary, dept_id
FROM employees e1
WHERE salary > (
    SELECT AVG(salary)
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id -- correlated reference
);
```

5.4 EXISTS / NOT EXISTS

```
-- Customers who placed at least one order
SELECT c.name
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);

-- Customers with NO orders
SELECT c.name
FROM customers c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.customer_id = c.customer_id
);
```

5.5 Derived Table (inline view)

```
SELECT dept_avg.dept_id, dept_avg.avg_sal
FROM (
    SELECT dept_id, AVG(salary) AS avg_sal
    FROM employees
```

```

        GROUP BY dept_id
    ) AS dept_avg
WHERE dept_avg.avg_sal > 70000;

```

5.6 Common Table Expressions (CTEs)

Single CTE:

```

WITH high_earners AS (
    SELECT * FROM employees WHERE salary > 90000
)
SELECT he.first_name, d.dept_name
FROM high_earners he
JOIN departments d ON he.dept_id = d.dept_id;

```

Multiple CTEs:

```

WITH dept_stats AS (
    SELECT dept_id, AVG(salary) AS avg_sal, COUNT(*) AS cnt
    FROM employees
    GROUP BY dept_id
),
big_depts AS (
    SELECT dept_id FROM dept_stats WHERE cnt >= 10
)
SELECT e.first_name, ds.avg_sal
FROM employees e
JOIN dept_stats ds ON e.dept_id = ds.dept_id
WHERE e.dept_id IN (SELECT dept_id FROM big_depts);

```

Recursive CTE (hierarchy / tree traversal):

```

WITH RECURSIVE org_chart AS (
    -- anchor: top-level employees (no manager)
    SELECT emp_id, first_name, manager_id, 0 AS level
    FROM employees WHERE manager_id IS NULL
    UNION ALL
    -- recursive: each employee's direct reports
    SELECT e.emp_id, e.first_name, e.manager_id, oc.level + 1
    FROM employees e
    JOIN org_chart oc ON e.manager_id = oc.emp_id
)
SELECT emp_id, first_name, level FROM org_chart ORDER BY level, emp_id;

```

6. Set Operations

```

-- UNION: all rows, duplicates removed
SELECT city FROM customers
UNION
SELECT city FROM suppliers;

```

```
-- UNION ALL: all rows, duplicates kept (faster)
SELECT product_id FROM orders_2023
UNION ALL
SELECT product_id FROM orders_2024;

-- INTERSECT: rows in BOTH result sets
SELECT customer_id FROM orders_2023
INTERSECT
SELECT customer_id FROM orders_2024;

-- EXCEPT (MINUS in Oracle): rows in first but NOT second
SELECT email FROM all_users
EXCEPT
SELECT email FROM unsubscribed;
```

NOTE: All branches must have the same number and compatible types of columns. Column names come from the first SELECT.

7. Aggregate Functions

```
SELECT
  COUNT(*) AS total_rows,
  COUNT(email) AS non_null_emails, -- ignores NULL
  COUNT(DISTINCT dept_id) AS unique_depts,
  SUM(salary) AS payroll,
  AVG(salary) AS mean_salary,
  MIN(salary) AS lowest,
  MAX(salary) AS highest,
  STDDEV(salary) AS std_dev,
  VARIANCE(salary) AS variance,
  GROUP_CONCAT(last_name ORDER BY last_name SEPARATOR ', ') AS names -- MySQL
FROM employees
WHERE hire_date >= '2022-01-01';
```

Conditional aggregation (pivoting pattern):

```
-- PostgreSQL
SELECT dept_id, COUNT(*) FILTER (WHERE salary > 80000) AS high_earners FROM employees
GROUP BY dept_id;

-- MySQL
SELECT dept_id, SUM(CASE WHEN salary > 80000 THEN 1 ELSE 0 END) AS high_earners_mysql
FROM employees GROUP BY dept_id;
```

8. String Functions

Commonly used string operations across SQL engines:

LOWER(email)	-- Converts to lowercase
UPPER(first_name)	-- Converts to uppercase
CONCAT(first_name, ' ', last_name)	-- Concatenates strings
CONCAT_WS(' ', first_name, last_name)	-- Concatenates with separator
LENGTH(last_name)	-- String length in bytes
CHAR_LENGTH(last_name)	-- String length in characters
SUBSTRING(email, 1, INSTR(email, '@')-1)	-- Extracts username from email
LEFT(first_name, 3)	-- Leftmost 3 characters
RIGHT(email, 4)	-- Rightmost 4 characters
TRIM(' hello ')	-- Trims spaces
REPLACE(phone, ' ', '')	-- Replaces character occurrences
LPAD(emp_id, 6, '0')	-- Pads string left side
REVERSE(last_name)	-- Reverses string
LOCATE('@', email)	-- Finds character position
FORMAT(salary, 2)	-- Formats number to currency style
REGEXP_REPLACE(notes, '<[^>]+>', '')	-- Removes HTML tags via Regex (MySQL 8+)

9. Date & Time Functions

```

SELECT
    NOW() AS current_datetime,
    CURDATE() AS today,
    CURRENT_DATE AS today_ansi,
    CURTIME() AS now_time,
    YEAR(hire_date) AS hire_year,
    MONTH(hire_date) AS hire_month,
    DAY(hire_date) AS hire_day,
    DAYNAME(hire_date) AS day_name,
    WEEKDAY(hire_date) AS weekday_num,
    QUARTER(hire_date) AS quarter,
    DATEDIFF(CURDATE(), hire_date) AS days_employed,
    TIMESTAMPDIFF(YEAR, hire_date, NOW()) AS years_employed,
    DATE_ADD(hire_date, INTERVAL 90 DAY) AS probation_end,
    DATE_SUB(NOW(), INTERVAL 1 YEAR) AS one_year_ago,
    DATE_FORMAT(hire_date, '%d %b %Y') AS formatted_date,
    TO_CHAR(hire_date, 'DD Mon YYYY') AS formatted_pg, -- PostgreSQL
    LAST_DAY(hire_date) AS month_end,
    STR_TO_DATE('25-12-2024', '%d-%m-%Y') AS parsed_date,
    EXTRACT(YEAR FROM hire_date) AS yr_ansi,
    UNIX_TIMESTAMP(hire_date) AS epoch,
    FROM_UNIXTIME(1700000000) AS from_epoch
FROM employees;

```

10. Numeric & Math Functions

```

SELECT
    ROUND(salary, -3) AS rounded_to_k,
    FLOOR(salary / 1000) AS k_floor,
    CEILING(salary / 1000) AS k_ceil,

```

```

TRUNCATE(3.14159, 2) AS truncated, -- Returns 3.14 (MySQL)
ABS(-500) AS absolute,
MOD(17, 5) AS remainder,           -- Returns 2
POWER(2, 10) AS two_to_ten,       -- Returns 1024
SQRT(144) AS sqrt_val,            -- Returns 12
LOG(salary) AS natural_log,
LOG10(salary) AS log_base10,
EXP(1) AS e_const,
PI() AS pi_val,
RAND() AS random_0_to_1,
SIGN(-99) AS sign_val,
GREATEST(10, 20, 5) AS greatest_val,
LEAST(10, 20, 5) AS least_val
FROM employees;

```

11. CASE WHEN & Conditional Expressions

```

-- Searched CASE
SELECT first_name, salary,
       CASE
           WHEN salary > 100000 THEN 'Executive'
           WHEN salary >= 80000 THEN 'Senior'
           WHEN salary >= 60000 THEN 'Mid-Level'
           ELSE 'Junior'
       END AS grade
FROM employees;

-- Simple CASE (equality check)
SELECT dept_id,
       CASE dept_id
           WHEN 1 THEN 'Engineering'
           WHEN 2 THEN 'Marketing'
           WHEN 3 THEN 'Finance'
           ELSE 'Other'
       END AS dept_label
FROM employees;

-- COALESCE: returns first non-NULL value
SELECT COALESCE(phone, mobile, email, 'No Contact') AS contact FROM employees;

-- NULLIF: returns NULL if both args equal (avoids divide-by-zero)
SELECT total_sales / NULLIF(total_visits, 0) AS conversion_rate FROM campaigns;

-- IF (MySQL only)
SELECT first_name, IF(salary > 80000, 'High', 'Standard') AS pay_tier FROM employees;

-- IIF (SQL Server)
SELECT first_name, IIF(salary > 80000, 'High', 'Standard') AS pay_tier FROM employees;

```

12. Window Functions (Analytics)

Window functions compute values across rows related to the current row without collapsing them like GROUP BY does.

12.1 Syntax Template

```
function_name([expression]) OVER (  
    [PARTITION BY column1, column2]  
    [ORDER BY column3 ASC | DESC]  
    [ROWS | RANGE BETWEEN frame_start AND frame_end]  
)
```

12.2 Ranking Functions

```
SELECT  
    first_name, dept_id, salary,  
    ROW_NUMBER() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS row_num,  
    RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS rnk,  
    DENSE_RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dense_rnk,  
    NTILE(4) OVER (ORDER BY salary DESC) AS quartile,  
    PERCENT_RANK() OVER (ORDER BY salary) AS pct_rank,  
    CUME_DIST() OVER (ORDER BY salary) AS cume_dist  
FROM employees;
```

RANK vs DENSE_RANK vs ROW_NUMBER example:

Given salaries: 100, 100, 80, 70

- **RANK**: 1, 1, 3, 4 (gap after tie)
- **DENSE_RANK**: 1, 1, 2, 3 (no gap)
- **ROW_NUMBER**: 1, 2, 3, 4 (always unique)

12.3 Aggregate Window Functions

```
SELECT  
    first_name, dept_id, salary,  
    SUM(salary) OVER () AS company_total,  
    SUM(salary) OVER (PARTITION BY dept_id) AS dept_total,  
    COUNT(*) OVER (PARTITION BY dept_id) AS dept_headcount,  
    AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg,  
    salary / SUM(salary) OVER (PARTITION BY dept_id) * 100 AS pct_of_dept,  
  
    -- Running total  
    SUM(salary) OVER (  
        PARTITION BY dept_id  
        ORDER BY hire_date  
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW  
    ) AS running_payroll,  
  
    -- 3-month moving average
```

```
AVG(revenue) OVER (
    ORDER BY month_date
    ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
) AS moving_avg_3m
FROM employees;
```

12.4 LAG & LEAD - Access Adjacent Rows

```
SELECT
    emp_id, hire_date, salary,
    LAG(salary, 1, 0) OVER (PARTITION BY dept_id ORDER BY hire_date) AS
    prev_employee_salary,
    LEAD(salary, 1, 0) OVER (PARTITION BY dept_id ORDER BY hire_date) AS
    next_employee_salary,
    salary - LAG(salary) OVER (ORDER BY hire_date) AS salary_diff
FROM employees;
```

12.5 FIRST_VALUE & LAST_VALUE

```
SELECT
    first_name, dept_id, salary,
    FIRST_VALUE(salary) OVER (
        PARTITION BY dept_id ORDER BY salary DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS dept_max_salary,
    LAST_VALUE(salary) OVER (
        PARTITION BY dept_id ORDER BY salary DESC
        ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
    ) AS dept_min_salary
FROM employees;
```

12.6 ROWS vs RANGE frame

ROWS: Physical rows. E.g., Last 7 physical rows:

```
SUM(amount) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)
```

RANGE: Logical frame (rows with the same ORDER BY value share a frame):

```
SUM(amount) OVER (ORDER BY date RANGE BETWEEN INTERVAL 7 DAY PRECEDING AND CURRENT
ROW)
```

13. Indexes & Query Performance

EXPLAIN to see query plan (MySQL):

```
EXPLAIN SELECT * FROM employees WHERE last_name = 'Smith';
EXPLAIN FORMAT JSON SELECT * FROM employees WHERE dept_id = 3;
```

EXPLAIN ANALYZE actually executes the query (PostgreSQL):

```
EXPLAIN ANALYZE SELECT * FROM employees WHERE salary > 80000;
```

Composite index tips: Put equality columns first, range columns last.

```
CREATE INDEX idx_dept_salary ON employees (dept_id, salary);
-- Good: WHERE dept_id = 3 AND salary > 60000
-- Good: WHERE dept_id = 3
-- Bad: WHERE salary > 60000 (skips leftmost prefix)
```

Covering index (all needed columns in index):

```
CREATE INDEX idx_cover ON employees (dept_id, salary, first_name, last_name);
SELECT first_name, last_name FROM employees WHERE dept_id = 1 ORDER BY salary;
-- This query is satisfied entirely from index (no table lookup)
```

Index on expression (PostgreSQL):

```
CREATE INDEX idx_lower_email ON employees (LOWER(email));
-- Enables optimization for: WHERE LOWER(email) = 'alice@example.com'
```

TIP: Avoid functions on indexed columns in WHERE clause. `WHERE YEAR(hire_date) = 2023` cannot use the index on `hire_date`. Instead, use: `WHERE hire_date BETWEEN '2023-01-01' AND '2023-12-31'`.

14. Transactions & ACID

Explicit transaction:

```
START TRANSACTION; -- or BEGIN or BEGIN TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE account_id = 1;
UPDATE accounts SET balance = balance + 500 WHERE account_id = 2;

-- If both succeed:
COMMIT;

-- If anything fails:
ROLLBACK;
```

Savepoints (partial rollback):

```
START TRANSACTION;
INSERT INTO orders (customer_id, total) VALUES (5, 200);
SAVEPOINT after_order;
INSERT INTO order_items VALUES (NULL, LAST_INSERT_ID(), 101, 2);

-- If item insert fails:
ROLLBACK TO SAVEPOINT after_order;
COMMIT; -- order is saved, item is not
```


Isolation levels (set per session):

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
-- Levels: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE
```

15. Views & Stored Procedures

Create view:

```
CREATE OR REPLACE VIEW emp_summary AS
SELECT e.emp_id, CONCAT(e.first_name, ' ', e.last_name) AS name, d.dept_name, e.salary
FROM employees e
JOIN departments d ON e.dept_id = d.dept_id;

-- Use view like a table
SELECT * FROM emp_summary WHERE salary > 80000;

-- Updatable view (single table, no aggregates)
UPDATE emp_summary SET salary = 90000 WHERE emp_id = 5;
```

Materialized view (PostgreSQL):

```
CREATE MATERIALIZED VIEW monthly_sales AS
SELECT DATE_TRUNC('month', order_date) AS month, SUM(amount) AS revenue
FROM orders GROUP BY 1;

REFRESH MATERIALIZED VIEW monthly_sales;
```

Stored procedure (MySQL):

```
DELIMITER $$
CREATE PROCEDURE raise_salaries(
    IN p_dept_id INT,
    IN p_pct DECIMAL(5,2),
    OUT p_updated INT
)
BEGIN
    UPDATE employees
    SET salary = salary * (1 + p_pct / 100)
    WHERE dept_id = p_dept_id;
    SET p_updated = ROW_COUNT();
END$$
DELIMITER ;

-- Calling the procedure
CALL raise_salaries(3, 10, @n);
SELECT @n AS rows_updated;
```

Stored function:

```

DELIMITER $$
CREATE FUNCTION annual_bonus (sal DECIMAL(10,2), yrs INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN ROUND(sal * 0.10 * LEAST(yrs, 5), 2);
END$$
DELIMITER ;

SELECT first_name, annual_bonus(salary, YEAR(NOW())-YEAR(hire_date)) AS bonus
FROM employees;

```

16. SQL Data Types Reference

Type	Description	Example
INT/INTEGER	4-byte integer (-2B to 2B)	age INT
BIGINT	8-byte integer	user_id BIGINT
TINYINT	1-byte integer (0-255 unsigned)	is_active TINYINT(1)
SMALLINT	2-byte integer	rating SMALLINT
DECIMAL(p,s)	Exact fixed-point numeric	price DECIMAL(10,2)
FLOAT/DOUBLE	Approximate floating-point	score DOUBLE
CHAR(n)	Fixed-length string	country_code CHAR(2)
VARCHAR(n)	Variable-length string	name VARCHAR(100)
TEXT/LONGTEXT	Large text (64KB/4GB)	description TEXT
DATE	YYYY-MM-DD	birth_date DATE
TIME	HH:MM:SS	shift_start TIME
DATETIME	YYYY-MM-DD HH:MM:SS	created_at DATETIME
TIMESTAMP	Auto UTC, auto-updates	updated_at TIMESTAMP
YEAR	4-digit year	model_year YEAR
BOOLEAN/BOOL	Alias for TINYINT(1)	is_admin BOOLEAN
ENUM(...)	One value from list	status ENUM('active', 'inactive')
SET(...)	Zero or more from list	tags SET('a','b','c')
JSON	Native JSON document	metadata JSON
BLOB/LONGBLOB	Binary large object	avatar BLOB
UUID / CHAR(36)	Universally unique identifier	id CHAR(36) DEFAULT (UUID())

PART 2 - 50 LEETCODE SQL SOLUTIONS

Each solution includes the problem description, table schema, and optimized SQL.

#175 - Combine Two Tables [Easy]

Schema: Person(personId PK, lastName, firstName) | Address(addressId PK, personId FK, city, state)

Problem: Report firstName, lastName, city, state for every person (include those without an address).

```
SELECT p.firstName, p.lastName, a.city, a.state
FROM Person p
LEFT JOIN Address a ON p.personId = a.personId;
```

Key Insight: LEFT JOIN ensures persons without addresses still appear (city/state = NULL).

#176 - Second Highest Salary [Easy]

Schema: Employee(id PK, salary)

Problem: Return the second distinct highest salary. Return NULL if it doesn't exist.

```
SELECT MAX(salary) AS SecondHighestSalary
FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);

-- Alternative using LIMIT/OFFSET
SELECT (
    SELECT DISTINCT salary
    FROM Employee
    ORDER BY salary DESC
    LIMIT 1 OFFSET 1
) AS SecondHighestSalary;
```

Key Insight: Wrapping in a subquery ensures NULL is returned instead of an empty result set.

#177 - Nth Highest Salary [Medium]

Schema: Employee(id, salary)

Problem: Create a function getNthHighestSalary (N INT) returning the Nth distinct highest salary.

```
CREATE FUNCTION getNthHighestSalary(N INT) RETURNS INT
BEGIN
    SET N = N - 1;
    RETURN (
        SELECT DISTINCT salary
        FROM Employee
        ORDER BY salary DESC
        LIMIT 1 OFFSET N
    );
END
```

#181 - Employees Earning More Than Their Managers [Easy]

Schema: Employee(id, name, salary, managerId)

Problem: Find employees who earn more than their manager.

```
SELECT e.name AS Employee
FROM Employee e
JOIN Employee m ON e.managerId = m.id
WHERE e.salary > m.salary;
```

Key Insight: Self-join the Employee table aliased as manager.

#182 - Duplicate Emails [Easy]

Schema: Person(id PK, email)

Problem: Find all emails that appear more than once.

```
SELECT email
FROM Person
GROUP BY email
HAVING COUNT(*) > 1;
```

#183 - Customers Who Never Order [Easy]

Schema: Customers(id, name) | Orders(id, customerId FK)

Problem: Find customers who have never placed an order.

```
SELECT name AS Customers
FROM Customers
WHERE id NOT IN (SELECT customerId FROM Orders);

-- Alternative (often faster)
SELECT c.name AS Customers
FROM Customers c
LEFT JOIN Orders o ON c.id = o.customerId
WHERE o.id IS NULL;
```

#184 - Department Highest Salary [Medium]

Schema: Employee(id, name, salary, departmentId) | Department(id, name)

Problem: Find employees with the highest salary in each department.

```
SELECT d.name AS Department, e.name AS Employee, e.salary AS Salary
FROM Employee e
JOIN Department d ON e.departmentId = d.id
WHERE (e.departmentId, e.salary) IN (
    SELECT departmentId, MAX(salary)
    FROM Employee
    GROUP BY departmentId
);
```

Key Insight: Row-value constructor (col1, col2) IN (subquery) is clean and idiomatic.

#185 - Department Top Three Salaries [Hard]

Schema: Employee(id, name, salary, departmentId) | Department(id, name)

Problem: Find employees in the top 3 unique salary brackets per department.

```
WITH ranked AS (  
    SELECT e.name AS Employee, e.salary, d.name AS Department,  
           DENSE_RANK() OVER (  
               PARTITION BY e.departmentId  
               ORDER BY e.salary DESC  
           ) AS rnk  
    FROM Employee e  
    JOIN Department d ON e.departmentId = d.id  
)  
SELECT Department, Employee, salary AS Salary  
FROM ranked  
WHERE rnk <= 3;
```

Key Insight: DENSE_RANK preserves ranks across ties, ensuring exactly the top-3 salary brackets.

#196 - Delete Duplicate Emails [Easy]

Schema: Person(id PK, email)

Problem: Delete all duplicate email entries keeping only the smallest id.

```
DELETE e1  
FROM Person e1  
JOIN Person e2  
ON e1.email = e2.email AND e1.id > e2.id;
```

Key Insight: Self-join deletes any row where a smaller-id duplicate exists.

#197 - Rising Temperature [Easy]

Schema: Weather(id, recordDate DATE, temperature)

Problem: Find ids of dates that were warmer than the previous day.

```
SELECT w1.id  
FROM Weather w1  
JOIN Weather w2  
ON DATEDIFF(w1.recordDate, w2.recordDate) = 1  
WHERE w1.temperature > w2.temperature;
```

Key Insight: DATEDIFF = 1 finds exactly the previous calendar day regardless of id order.

#262 - Trips and Users [Hard]

Schema: Trips(id, client_id, driver_id, city_id, status, request_at DATE) | Users(users_id, banned, role)

Problem: Compute cancellation rate (rounded to 2 dp) for non-banned users, Oct 1-3 2013.

```

SELECT request_at AS Day,
       ROUND(SUM(status != 'completed') / COUNT(*), 2) AS "Cancellation Rate"
FROM Trips
WHERE request_at BETWEEN '2013-10-01' AND '2013-10-03'
      AND client_id NOT IN (SELECT users_id FROM Users WHERE banned = 'Yes')
      AND driver_id NOT IN (SELECT users_id FROM Users WHERE banned = 'Yes')
GROUP BY request_at;

```

Key Insight: status != 'completed' returns 1 (true) for cancelled trips in MySQL, enabling SUM as count.

#511 - Game Play Analysis I [Easy]

Schema: Activity(player_id, device_id, event_date DATE, games_played)

Problem: Find the first login date for each player.

```

SELECT player_id, MIN(event_date) AS first_login
FROM Activity
GROUP BY player_id;

```

#512 - Game Play Analysis II [Easy]

Schema: Activity(player_id, device_id, event_date DATE, games_played)

Problem: Report device used on each player's first login day.

```

SELECT player_id, device_id
FROM Activity
WHERE (player_id, event_date) IN (
    SELECT player_id, MIN(event_date)
    FROM Activity
    GROUP BY player_id
);

```

#550 - Game Play Analysis IV [Medium]

Schema: Activity(player_id, device_id, event_date DATE, games_played)

Problem: Fraction of players who logged in the day after their first login (round to 2 dp).

```

SELECT ROUND(COUNT(DISTINCT a2.player_id) / COUNT(DISTINCT a1.player_id), 2) AS
fraction
FROM Activity a1
LEFT JOIN Activity a2 ON a1.player_id = a2.player_id
      AND DATEDIFF(a2.event_date, a1.event_date) = 1
WHERE a1.event_date = (
    SELECT MIN(event_date) FROM Activity a3 WHERE a3.player_id = a1.player_id
);

```

#570 - Managers with at Least 5 Direct Reports [Medium]

Schema: Employee(id, name, department, managerId)

Problem: Find managers who have at least 5 direct reports.

```
SELECT name
FROM Employee
WHERE id IN (
    SELECT managerId
    FROM Employee
    GROUP BY managerId
    HAVING COUNT(*) >= 5
);
```

#571 - Find Median Given Frequency of Numbers [Hard]

Schema: Numbers(num INT, frequency INT)

Problem: Find the median of the number set described by the frequency table.

```
WITH cumulative AS (
    SELECT num, frequency,
           SUM(frequency) OVER (ORDER BY num) AS cum_freq,
           SUM(frequency) OVER () AS total
    FROM Numbers
)
SELECT AVG(CAST(num AS FLOAT)) AS median
FROM cumulative
WHERE cum_freq >= total / 2.0
    AND cum_freq - frequency <= (total + 1) / 2.0;
```

Key Insight: The median lies in rows where the cumulative frequency window straddles the midpoint.

#574 - Winning Candidate [Medium]

Schema: Candidate(id, name) | Vote(id, candidateId)

Problem: Find the name of the winning candidate (most votes).

```
SELECT c.name
FROM Candidate c
JOIN Vote v ON c.id = v.candidateId
GROUP BY c.id, c.name
ORDER BY COUNT(*) DESC
LIMIT 1;
```

#577 - Employee Bonus [Easy]

Schema: Employee(empId PK, name, supervisor, salary) | Bonus(empId PK FK, bonus)

Problem: Report name and bonus for employees with bonus < 1000 or no bonus entry.

```
SELECT e.name, b.bonus
FROM Employee e
LEFT JOIN Bonus b ON e.empId = b.empId
WHERE b.bonus < 1000 OR b.bonus IS NULL;
```

#578 - Get Highest Answer Rate Question [Medium]

Schema: SurveyLog(uid, action ENUM('show', 'answer', 'skip'), question_id, answer_id, q_num, timestamp)

Problem: Find the question with the highest answer rate.

```
SELECT question_id
FROM SurveyLog
GROUP BY question_id
ORDER BY SUM(action='answer') / SUM(action='show') DESC
LIMIT 1;
```

#579 - Find Cumulative Salary of an Employee [Hard]

Schema: Employee(id, month INT, salary INT)

Problem: For each employee, for each month (excluding most recent), sum of last 3 months' salaries.

```
SELECT id, month,
       SUM(salary) OVER (
         PARTITION BY id
         ORDER BY month
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ) AS Salary
FROM Employee
WHERE (id, month) NOT IN (
  SELECT id, MAX(month) FROM Employee GROUP BY id
)
ORDER BY id, month DESC;
```

Key Insight: Sliding window of 3 rows (2 preceding + current) gives rolling 3-month sum.

#580 - Count Student Number in Departments [Medium]

Schema: Department(dept_id PK, dept_name) | Student(student_id PK, student_name, gender, dept_id FK)

Problem: Report department name and student count, ordered by count desc then name.

```
SELECT d.dept_name, COUNT(s.student_id) AS student_number
FROM Department d
LEFT JOIN Student s ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY student_number DESC, d.dept_name;
```


#584 - Find Customer Referee [Easy]

Schema: Customer(id, name, referee_id)

Problem: Find customers not referred by customer with id=2.

```
SELECT name
FROM Customer
WHERE referee_id != 2 OR referee_id IS NULL;
```

Key Insight: referee_id != 2 alone would exclude NULLs; must add OR IS NULL.

#585 - Investments in 2016 [Medium]

Schema: Insurance(pid PK, tiv_2015, tiv_2016, lat, lon)

Problem: Sum tiv_2016 (2dp) for policies with same tiv_2015 as another policy and unique (lat, lon).

```
SELECT ROUND(SUM(tiv_2016), 2) AS tiv_2016
FROM Insurance
WHERE tiv_2015 IN (
    SELECT tiv_2015 FROM Insurance GROUP BY tiv_2015 HAVING COUNT(*) > 1
) AND (lat, lon) IN (
    SELECT lat, lon FROM Insurance GROUP BY lat, lon HAVING COUNT(*) = 1
);
```

#586 - Customer Placing the Largest Number of Orders [Easy]

Schema: Orders(order_number PK, customer_number)

Problem: Find the customer_number with the most orders.

```
SELECT customer_number
FROM Orders
GROUP BY customer_number
ORDER BY COUNT(*) DESC
LIMIT 1;
```

#595 - Big Countries [Easy]

Schema: World(name, continent, area, population, gdp)

Problem: Find countries with area >= 3M km² OR population >= 25M.

```
SELECT name, population, area
FROM World
WHERE area >= 3000000 OR population >= 25000000;
```

#596 - Classes with More than 5 Students [Easy]

Schema: Courses(student VARCHAR, class VARCHAR)

Problem: Find all classes with at least 5 students.

```
SELECT class
FROM Courses
GROUP BY class
HAVING COUNT(DISTINCT student) >= 5;
```

#597 - Friend Requests I: Overall Acceptance Rate [Easy]

Schema: FriendRequest(sender_id, send_to_id, request_date) | RequestAccepted(requester_id, acceptor_id, accept_date)

Problem: Compute overall friend-request acceptance rate (round to 2dp, 0.00 if none).

```
SELECT ROUND(
    IFNULL(
        (SELECT COUNT(DISTINCT requester_id, acceptor_id) FROM RequestAccepted) /
        (SELECT COUNT(DISTINCT sender_id, send_to_id) FROM FriendRequest),
        0), 2) AS accept_rate;
```

#601 - Human Traffic of Stadium [Hard]

Schema: Stadium(id, visit_date DATE, people INT)

Problem: Find rows with >=100 people that are part of 3+ consecutive id rows all with >=100 people.

```
WITH filtered AS (
    SELECT *, ROW_NUMBER() OVER (ORDER BY id) AS rn
    FROM Stadium WHERE people >= 100
),
grouped AS (
    SELECT *, (id - rn) AS grp FROM filtered
)
SELECT id, visit_date, people
FROM grouped
WHERE grp IN (SELECT grp FROM grouped GROUP BY grp HAVING COUNT(*) >= 3)
ORDER BY visit_date;
```

Key Insight: id - ROW_NUMBER() is constant for consecutive id sequences - a classic gaps-and-islands trick.

#602 - Friend Requests II: Who Has the Most Friends [Medium]

Schema: RequestAccepted(requester_id, acceptor_id, accept_date)

Problem: Find the person with the most friends (both directions count).

```
WITH all_pairs AS (
    SELECT requester_id AS id FROM RequestAccepted
    UNION ALL
    SELECT acceptor_id AS id FROM RequestAccepted
)
SELECT id, COUNT(*) AS num
FROM all_pairs
GROUP BY id
```

```
ORDER BY num DESC
LIMIT 1;
```

#607 - Sales Person [Easy]

Schema: SalesPerson(sales_id PK, name, ...) | Company(com_id PK, name, city) | Orders(order_id PK, order_date, com_id FK, sales_id FK, amount)

Problem: Find salespersons who have NO order with company 'RED'.

```
SELECT name
FROM SalesPerson
WHERE sales_id NOT IN (
    SELECT o.sales_id
    FROM Orders o
    JOIN Company c ON o.com_id = c.com_id
    WHERE c.name = 'RED'
);
```

#608 - Tree Node [Medium]

Schema: Tree(id, p_id)

Problem: Classify each node as Root, Inner, or Leaf.

```
SELECT id,
CASE
    WHEN p_id IS NULL THEN 'Root'
    WHEN id IN (SELECT DISTINCT p_id FROM Tree WHERE p_id IS NOT NULL) THEN
'Inner'
    ELSE 'Leaf'
END AS type
FROM Tree;
```

#610 - Triangle Judgement [Easy]

Schema: Triangle(x, y, z)

Problem: For each row report whether (x,y,z) can form a triangle.

```
SELECT x, y, z,
CASE WHEN x+y>z AND x+z>y AND y+z>x THEN 'Yes' ELSE 'No' END AS triangle
FROM Triangle;
```

#612 - Shortest Distance in a Plane [Medium]

Schema: Point2D(x INT, y INT, PK(x,y))

Problem: Find the shortest Euclidean distance between any two points (round to 2dp).

```
SELECT ROUND(
    MIN(SQRT(POWER(p1.x - p2.x, 2) + POWER(p1.y - p2.y, 2))),
2) AS shortest
```

```
FROM Point2D p1
JOIN Point2D p2 ON (p1.x, p1.y) < (p2.x, p2.y);
```

Key Insight: $(p1.x, p1.y) < (p2.x, p2.y)$ avoids counting each pair twice and eliminates self-pairing.

#613 - Shortest Distance in a Line [Easy]

Schema: Point(x INT PK)

Problem: Find minimum distance between any two points on a line.

```
SELECT MIN(ABS(p1.x - p2.x)) AS shortest
FROM Point p1
JOIN Point p2 ON p1.x != p2.x;
```

#614 - Second Degree Follower [Medium]

Schema: Follow(followee VARCHAR, follower VARCHAR)

Problem: Find followers who are also followees, with their follower count.

```
SELECT f1.follower, COUNT(DISTINCT f2.follower) AS num
FROM Follow f1
JOIN Follow f2 ON f1.follower = f2.followee
GROUP BY f1.follower
ORDER BY f1.follower;
```

#615 - Average Salary: Departments vs Company [Hard]

Schema: Salary(id, employee_id, amount, pay_date DATE) | Employee(employee_id, department_id)

Problem: For each month compare each department's avg salary to the company avg. Return 'higher', 'lower', 'same'.

```
SELECT
    DATE_FORMAT(s.pay_date, '%Y-%m') AS pay_month,
    e.department_id,
    CASE
        WHEN AVG(s.amount) > AVG(AVG(s.amount)) OVER (PARTITION BY
DATE_FORMAT(s.pay_date, '%Y-%m')) THEN 'higher'
        WHEN AVG(s.amount) < AVG(AVG(s.amount)) OVER (PARTITION BY
DATE_FORMAT(s.pay_date, '%Y-%m')) THEN 'lower'
        ELSE 'same'
    END AS comparison
FROM Salary s
JOIN Employee e ON s.employee_id = e.employee_id
GROUP BY pay_month, e.department_id;
```

Key Insight: Nested aggregate: inner AVG groups by dept, outer AVG() OVER gives company avg for that month.

#618 - Students Report by Geography [Hard]

Schema: Student(name VARCHAR, continent ENUM('America', 'Asia', 'Europe'))

Problem: Pivot: one column per continent, row-by-row alphabetical, NULL for shorter continents.

```
WITH ranked AS (  
    SELECT name, continent,  
           ROW_NUMBER() OVER (PARTITION BY continent ORDER BY name) AS rn  
    FROM Student  
)  
SELECT  
    MAX(CASE WHEN continent = 'America' THEN name END) AS America,  
    MAX(CASE WHEN continent = 'Asia' THEN name END) AS Asia,  
    MAX(CASE WHEN continent = 'Europe' THEN name END) AS Europe  
FROM ranked  
GROUP BY rn;
```

Key Insight: ROW_NUMBER per continent + GROUP BY rn effectively zips the three ordered lists.

#619 - Biggest Single Number [Easy]

Schema: MyNumbers(num INT)

Problem: Find the largest number appearing exactly once; NULL if none.

```
SELECT MAX(num) AS num  
FROM MyNumbers  
GROUP BY num  
HAVING COUNT(*) = 1;
```

#620 - Not Boring Movies [Easy]

Schema: Cinema(id, movie, description, rating FLOAT)

Problem: Find movies with odd id and description not 'boring', sorted by rating desc.

```
SELECT * FROM Cinema  
WHERE id % 2 = 1 AND description != 'boring'  
ORDER BY rating DESC;
```

#626 - Exchange Seats [Medium]

Schema: Seat(id INT, student VARCHAR)

Problem: Swap every two adjacent students by id. If last student is odd-id, leave in place.

```
SELECT id,  
       CASE  
           WHEN id % 2 = 1 AND id < (SELECT COUNT(*) FROM Seat) THEN (SELECT student FROM  
Seat WHERE id = s.id + 1)  
           WHEN id % 2 = 0 THEN (SELECT student FROM Seat WHERE id = s.id - 1)  
           ELSE student  
       END AS student
```

```

FROM Seat s
ORDER BY id;

-- Cleaner window-function approach
SELECT id,
       CASE
         WHEN MOD(id, 2) = 0 THEN LAG(student) OVER (ORDER BY id)
         ELSE LEAD(student, 1, student) OVER (ORDER BY id)
       END AS student
FROM Seat;

```

#627 - Swap Salary [Easy]

Schema: Salary(id, name, sex ENUM('m','f'), salary)

Problem: Swap all 'm' and 'f' values in sex column in a single UPDATE, no temp table.

```

UPDATE Salary
SET sex = CASE sex WHEN 'm' THEN 'f' ELSE 'm' END;

-- Alternative with IF
UPDATE Salary SET sex = IF(sex='m', 'f', 'm');

```

#1045 - Customers Who Bought All Products [Medium]

Schema: Customer(customer_id, product_key) | Product(product_key PK)

Problem: Find customers who purchased every product in the Product table.

```

SELECT customer_id
FROM Customer
GROUP BY customer_id
HAVING COUNT(DISTINCT product_key) = (SELECT COUNT(*) FROM Product);

```

Key Insight: Count of distinct products purchased must equal total product count.

#1050 - Actors and Directors Who Cooperated At Least Three Times [Easy]

Schema: ActorDirector(actor_id, director_id, timestamp)

Problem: Find (actor_id, director_id) pairs with at least 3 joint projects.

```

SELECT actor_id, director_id
FROM ActorDirector
GROUP BY actor_id, director_id
HAVING COUNT(*) >= 3;

```

#1068 - Product Sales Analysis I [Easy]

Schema: Sales(sale_id, product_id FK, year, quantity, price) | Product(product_id PK, product_name)

Problem: Report product_name, year, price for every entry in Sales.

```
SELECT p.product_name, s.year, s.price
FROM Sales s
JOIN Product p ON s.product_id = p.product_id;
```

#1069 - Product Sales Analysis II [Easy]

Schema: Same as 1068

Problem: Report total quantity sold per product.

```
SELECT product_id, SUM(quantity) AS total_quantity
FROM Sales
GROUP BY product_id;
```

#1070 - Product Sales Analysis III [Medium]

Schema: Same as 1068

Problem: For each product, select the first year of sale, along with quantity and price.

```
SELECT product_id, year AS first_year, quantity, price
FROM Sales
WHERE (product_id, year) IN (
    SELECT product_id, MIN(year) FROM Sales GROUP BY product_id
);
```

#1075 - Project Employees I [Easy]

Schema: Project(project_id FK, employee_id FK) | Employee(employee_id PK, name, experience_years INT)

Problem: Report average experience years per project, rounded to 2dp.

```
SELECT project_id, ROUND(AVG(e.experience_years), 2) AS average_years
FROM Project p
JOIN Employee e ON p.employee_id = e.employee_id
GROUP BY project_id;
```

#1076 - Project Employees II [Easy]

Schema: Same as 1075

Problem: Find the project(s) with the most employees.

```
SELECT project_id
FROM Project
GROUP BY project_id
HAVING COUNT(*) = (
    SELECT COUNT(*) FROM Project GROUP BY project_id ORDER BY COUNT(*) DESC LIMIT 1
);
```

#1082 - Sales Analysis I [Easy]

Schema: Product(product_id PK, product_name, unit_price) | Sales(seller_id, product_id FK, buyer_id, sale_date, units, price)

Problem: Report the seller(s) with the highest total sale price.

```
SELECT seller_id
FROM Sales
GROUP BY seller_id
HAVING SUM(price) = (
    SELECT SUM(price) AS total FROM Sales GROUP BY seller_id ORDER BY total DESC LIMIT
1
);
```

#1083 - Sales Analysis II [Easy]

Schema: Same as 1082

Problem: Find buyers who bought product S8 but not iPhone.

```
SELECT DISTINCT buyer_id
FROM Sales s
JOIN Product p ON s.product_id = p.product_id
WHERE p.product_name = 'S8'
AND s.buyer_id NOT IN (
    SELECT s2.buyer_id
    FROM Sales s2
    JOIN Product p2 ON s2.product_id = p2.product_id
    WHERE p2.product_name = 'iPhone'
);
```

Quick-Reference: Common Patterns & Best Practices

De-duplicate rows:

```
SELECT DISTINCT col1, col2 FROM tbl;

-- Or keep first occurrence and delete duplicates
DELETE FROM tbl WHERE id NOT IN (
    SELECT MIN(id) FROM tbl GROUP BY email
);
```

Top-N per group:

```
WITH ranked_tbl AS (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY group_col ORDER BY rank_col DESC) AS rn
    FROM tbl
)
SELECT * FROM ranked_tbl WHERE rn <= N;
```

Running total:


```
SUM(amount) OVER (ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)
```

Pivot (rows → columns):

```
SELECT year,  
       SUM(CASE WHEN month = 1 THEN sales END) AS Jan,  
       SUM(CASE WHEN month = 2 THEN sales END) AS Feb,  
       SUM(CASE WHEN month = 3 THEN sales END) AS Mar  
FROM monthly_sales  
GROUP BY year;
```

Unpivot (columns → rows):

```
SELECT year, 'Q1' AS quarter, q1 AS sales FROM t  
UNION ALL SELECT year, 'Q2', q2 FROM t  
UNION ALL SELECT year, 'Q3', q3 FROM t;
```

Consecutive rows / Islands:

```
-- Find gaps using id - ROW_NUMBER()  
SELECT MIN(id), MAX(id), COUNT(*) AS streak  
FROM (  
    SELECT id, id - ROW_NUMBER() OVER (ORDER BY id) AS grp  
    FROM tbl  
) x  
GROUP BY grp;
```

Year-over-year growth:

```
SELECT year, revenue,  
       LAG(revenue) OVER (ORDER BY year) AS prev_year,  
       ROUND((revenue - LAG(revenue) OVER (ORDER BY year)) / LAG(revenue) OVER (ORDER BY  
year) * 100, 1) AS yoy_pct  
FROM annual_revenue;
```

Median in MySQL (no built-in MEDIAN):

```
SELECT AVG(salary) AS median FROM (  
    SELECT salary FROM employees ORDER BY salary  
) t  
LIMIT 2 - (SELECT COUNT(*) FROM employees) % 2  
OFFSET FLOOR((SELECT COUNT(*) FROM employees) / 2);
```